

Muitas aplicações exigem um conjunto dinâmico que suporte somente as operações de dicionário INSERT, SEARCH e DELETE. Por exemplo, um compilador que traduz uma linguagem de programação mantém uma tabela de símbolos na qual as chaves de elementos são cadeias de caracteres arbitrários que correspondem a identificadores na linguagem. Uma tabela de espalhamento ou hashing é uma estrutura de dados eficaz para implementar dicionários. Embora a busca por um elemento em uma tabela de espalhamento possa demorar tanto quanto procurar um elemento em uma lista ligada o tempo $\Theta(n)$ no pior caso, na prática o hashing funciona extremamente bem. Sob premissas razoáveis, o tempo médio para pesquisar um elemento em uma tabela de espalhamento é $O(1)$.

Uma tabela de espalhamento generaliza a noção mais simples de um arranjo comum. O endereçamento direto em um arranjo comum faz uso eficiente de nossa habilidade de examinar uma posição arbitrária em um arranjo no tempo $O(1)$. A Seção 11.1 discute o endereçamento direto com mais detalhes. Podemos tirar proveito do endereçamento direto quando temos condições de alocar um arranjo que tem uma única posição para cada chave possível.

Quando o número de chaves realmente armazenadas é pequeno em relação ao número total de chaves possíveis, as tabelas de espalhamento se tornam uma alternativa eficaz para endereçar diretamente um arranjo, já que normalmente uma tabela de espalhamento utiliza um arranjo de tamanho proporcional ao número de chaves realmente armazenadas. Em vez de usar a chave diretamente como um índice de arranjo, o índice de arranjo é *computado* a partir da chave. A Seção 11.2 apresenta as principais ideias, focalizando o “encadeamento” como um modo de tratar “colisões”, nas quais mais de uma chave é mapeada para o mesmo índice de arranjo. A Seção 11.3 descreve como podemos computar os índices de arranjos a partir das chaves com o uso de funções hash. Apresentamos e analisamos diversas variações sobre o tema básico. A Seção 11.4 examina o “endereçamento aberto”, que é um outro modo de lidar com colisões. A conclusão é que o hash é uma técnica extremamente eficaz e prática: as operações básicas de dicionário exigem apenas o tempo $O(1)$ em média. A Seção 11.5 explica como o “hashing perfeito” pode suportar buscas no tempo do *pior caso* $O(1)$, quando o conjunto de chaves que está sendo armazenado é estático (isto é, quando o conjunto de chaves nunca muda uma vez armazenado).

11.1 TABELAS DE ENDEREÇO DIRETO

O endereçamento direto é uma técnica simples que funciona bem quando o universo U de chaves é razoavelmente pequeno. Suponha que uma aplicação necessite de um conjunto dinâmico no qual cada elemento tem uma chave extraída do universo $U = \{0, 1, \dots, m-1\}$, onde m não é muito grande. Consideramos que não há dois elementos com a mesma chave.

Para representar o conjunto dinâmico, usamos um arranjo, ou uma **tabela de endereços diretos**, denotada por $T[0 \dots m-1]$, na qual cada **posição, ou lacuna**, corresponde a uma chave no universo U . A Figura 11.1 ilustra a abordagem; a posição k aponta para um elemento no conjunto com chave k . Se o conjunto não contém nenhum elemento com chave k , então $T[k] = \text{NIL}$.

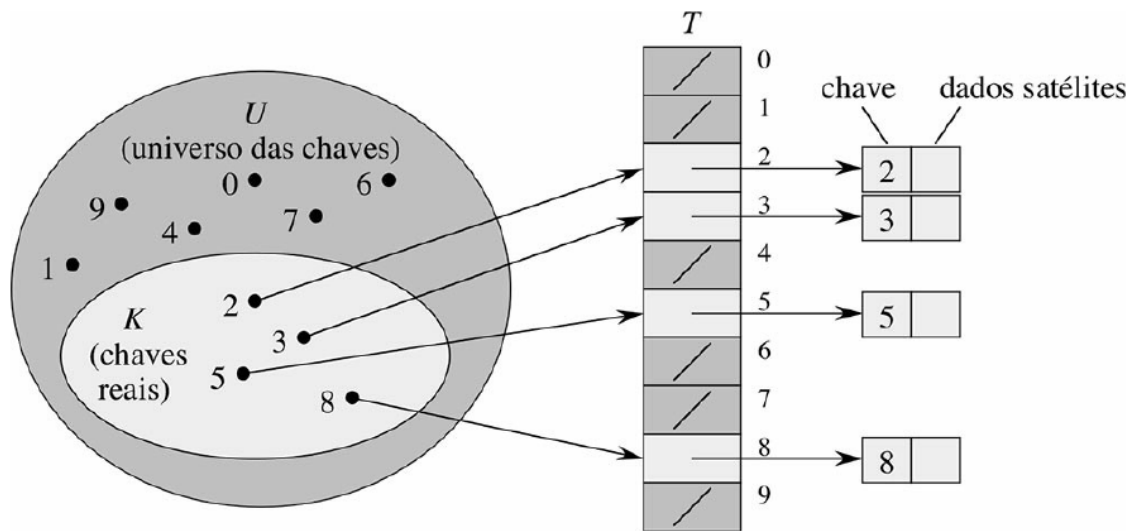


Figura 11.1 Como implementar um conjunto dinâmico por uma tabela de endereços diretos T . Cada chave no universo $U = \{0, 1, \dots, 9\}$ corresponde a um índice na tabela. O conjunto $K = \{2, 3, 5, 8\}$ de chaves reais determina as posições na tabela que contêm ponteiros para elementos. As outras posições, sombreadas em tom mais escuro, contêm NIL.

A implementação das operações de dicionário é trivial.

DIRECT-ADDRESS-SEARCH(T, k)

1 **return** $T[k] = x$

DIRECT-ADDRESS-INSERT(T, x)

1 $T[x.chave] = x$

DIRECT-ADDRESS-DELETE(T, x)

1 $T[x.chave] = \text{NIL}$

Cada uma dessas operações leva somente o tempo $O(1)$.

Em algumas aplicações, a própria tabela de endereços diretos pode conter os elementos do conjunto dinâmico. Isto é, em vez de armazenar a chave e os dados satélites de um elemento em um objeto externo à tabela de endereços diretos, com um ponteiro de uma posição na tabela até o objeto, podemos armazenar o objeto na própria posição e, portanto, economizar espaço. Usaríamos uma chave especial dentro de um objeto para indicar uma lacuna. Além disso, muitas vezes, é desnecessário armazenar a chave do objeto, já que, se temos o índice de um objeto na tabela, temos sua chave. Entretanto, se as chaves não forem armazenadas, devemos ter algum modo de saber se a posição está vazia.

Exercícios

11.1-1 Suponha que um conjunto dinâmico S seja representado por uma tabela de endereços diretos T de comprimento m . Descreva um procedimento que determine o elemento máximo de S . Qual é o desempenho do pior caso do seu procedimento?

11.1-2 Um **vetor de bits** é simplesmente um arranjo de bits (0s e 1s). Um vetor de bits de comprimento m ocupa um espaço muito menor que um arranjo de m ponteiros. Descreva como usar um vetor de bits para representar um conjunto dinâmico de elementos distintos sem dados satélites. Operações de dicionário devem ser executadas no tempo $O(1)$.

11.1-3 Sugira como implementar uma tabela de endereços diretos na qual as chaves de elementos armazenados não precisem ser distintas e os elementos possam ter dados satélites. Todas as três operações de dicionário (INSERT, DELETE e SEARCH) devem ser executadas no tempo $O(1)$. (Não esqueça que DELETE adota como argumento um ponteiro para um objeto a ser eliminado, não uma chave.)

11.1-4 ★ Desejamos implementar um dicionário usando endereçamento direto para um arranjo *enorme*. No início, as entradas do arranjo podem conter lixo, e inicializar o arranjo inteiro é impraticável devido ao seu tamanho. Descreva um esquema para implementar um dicionário de endereço direto para um arranjo enorme. Cada objeto armazenado deve utilizar espaço $O(1)$; as operações SEARCH, INSERT e DELETE devem demorar tempo $O(1)$ cada uma e a inicialização da estrutura de dados deve demorar o tempo $O(1)$. (*Sugestão*: Use um arranjo adicional tratado de certo modo como uma pilha cujo tamanho é o número de chaves realmente armazenadas no dicionário, para ajudar a determinar se uma dada entrada no arranjo enorme é válida ou não.)

11.2 TABELAS DE ESPALHAMENTO

O aspecto negativo do endereçamento direto é óbvio: se o universo U é grande, armazenar uma tabela T de tamanho $|U|$ pode ser impraticável ou mesmo impossível, dada a memória disponível em um computador típico. Além disso, o conjunto K de chaves *realmente armazenadas* pode ser tão pequeno em relação a U que grande parte do espaço alocado para T seria desperdiçada.

Quando o conjunto K de chaves armazenadas em um dicionário é muito menor que o universo U de todas as chaves possíveis, uma tabela de espalhamento requer armazenamento muito menor que uma tabela de endereços diretos. Especificamente, podemos reduzir o requisito de armazenamento a $(|K|)$ e ao mesmo tempo manter o benefício de procurar um elemento na tabela ainda no tempo $O(1)$. A pegada é que esse limite é para o *caso do tempo médio*, enquanto no caso do endereçamento direto ele vale para o *tempo do pior caso*.

Com endereçamento direto, um elemento com a chave k é armazenado na posição k . Com hash, esse elemento é armazenado na posição $h(k)$; isto é, usamos uma **função hash** h para calcular a posição pela chave k . Aqui, h mapeia o universo U de chaves para as posições de uma **tabela de espalhamento** $T[0 .. m - 1]$:

$$h : U \rightarrow \{0, 1, \dots, m - 1\} ,$$

onde o tamanho m da tabela de espalhamento normalmente é muito menor que $|U|$. Dizemos que um elemento com a chave k **se espalha** (hashes) até a posição $h(k)$; dizemos também que $h(k)$ é o **valor hash** da chave k . A Figura 11.2 ilustra a ideia básica. A função hash reduz a faixa de índices do arranjo e, conseqüentemente, o tamanho do arranjo. Em vez de ter tamanho $|U|$, o arranjo pode ter tamanho m .

Porém há um revés: após o hash, duas chaves podem ser mapeadas para a mesma posição. Chamamos essa situação de **colisão**. Felizmente, existem técnicas eficazes para resolver o conflito criado por colisões.

É claro que a solução ideal seria evitar por completo as colisões. Poderíamos tentar alcançar essa meta escolhendo uma função hash adequada h . Uma ideia é fazer h parecer “aleatória”, evitando assim as colisões ou ao menos minimizando seu número. A expressão “to hash”, em inglês, que evoca imagens de misturas e retalhamentos aleatórios, capta o espírito dessa abordagem. (É claro que uma função hash h deve ser determinística, no sentido de que uma dada entrada k sempre deve produzir a mesma saída $h(k)$.) Porém, como $|U| > m$, devem existir no mínimo duas chaves que têm o mesmo valor hash; portanto, evitar totalmente as colisões é impossível. Assim, embora uma função hash bem projetada e de aparência “aleatória” possa minimizar o número de colisões, ainda precisamos de um método para resolver as colisões que ocorrerem.

O restante desta seção apresenta a técnica mais simples para resolução de colisões, denominada encadeamento. A Seção 11.4 apresenta um método alternativo para resolver colisões, denominado endereçamento aberto.

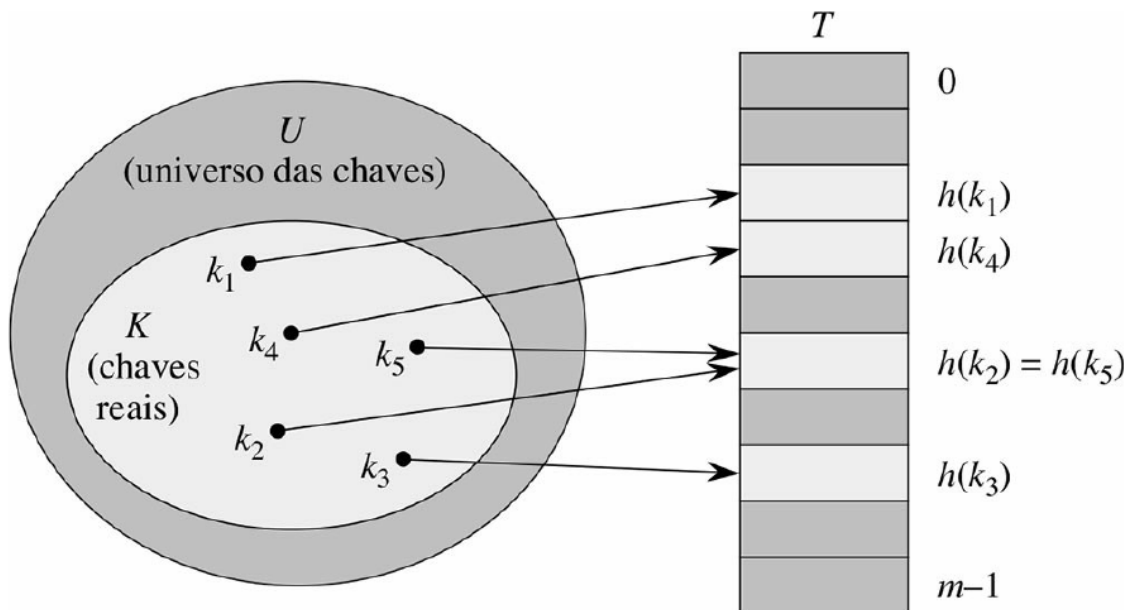


Figura 11.2 Utilização de uma função hash h para mapear chaves para posições de uma tabela de espalhamento. Como são mapeadas para a mesma posição, as chaves k_2 e k_5 colidem.

Resolução de colisões por encadeamento

No **encadeamento**, todos os elementos resultantes do hash vão para a mesma posição em uma lista ligada, como mostra a Figura 11.3. A posição j contém um ponteiro para o início da lista de todos os elementos armazenados que, após o hash, foram para j ; se não houver nenhum desses elementos, a posição j contém **NIL**.

As operações de dicionário em uma tabela de espalhamento T são fáceis de implementar quando as colisões são resolvidas por encadeamento.

CHAINED-HASH-INSERT(T, x)

1 insere x no início da lista $T[h(x.chave)]$

CHAINED-HASH-SEARCH(T, k)

1 procura um elemento com a chave k na lista $T[h(k)]$

CHAINED-HASH-DELETE(T, x)

1 elimina x da lista $T[h(x.chave)]$

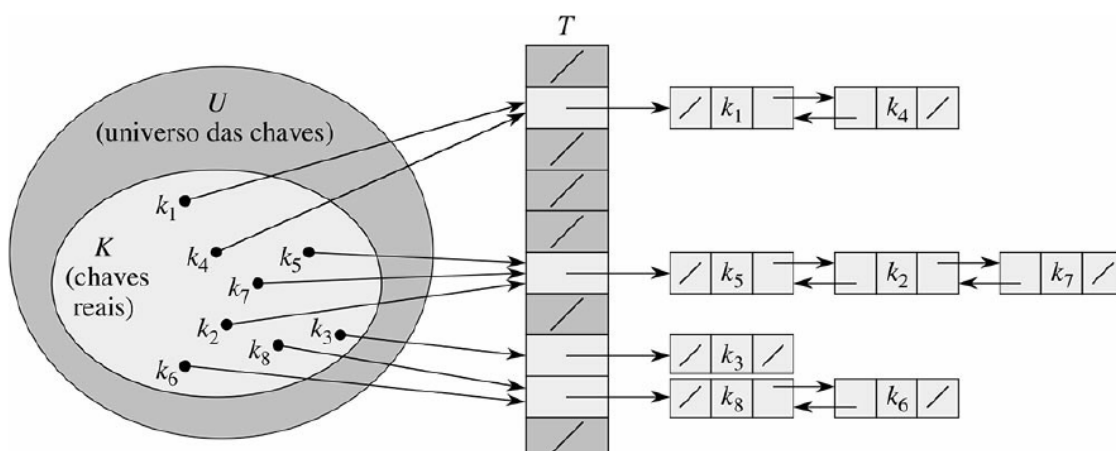


Figura 11.3 Resolução de colisão por encadeamento. Cada posição T_j da tabela de espalhamento contém uma lista ligada de todas as chaves cujo valor hash é j . Por exemplo, $h(k_1) = h(k_4)$ e $h(k_5) = h(k_2) = h(k_7)$. A lista ligada pode ser simplesmente ligada ou duplamente ligada; aqui ela é mostrada como duplamente ligada porque assim a eliminação é mais rápida.

O tempo de execução do pior caso para inserção é $O(1)$. O procedimento de inserção é rápido em parte porque supõe que o elemento x que está sendo inserido ainda não está presente na tabela; se necessário, podemos verificar essa premissa (a custo adicional) procurando um elemento cuja chave é $x.chave$ antes da inserção. Para busca, o tempo de execução do pior caso é proporcional ao comprimento da lista; analisaremos essa operação mais atentamente em seguida. Podemos eliminar um elemento x no tempo $O(1)$ se as listas forem duplamente ligadas, como mostra a Figura 11.3. (Observe que CHAINED-HASH-DELETE toma como entrada um elemento x e não sua chave k , de modo que não temos de procurar x antes. Se a tabela de espalhamento suportar eliminação, então suas listas ligadas devem ser duplamente ligadas para podermos eliminar um item rapidamente. Se as listas forem apenas simplesmente ligadas, para eliminar o elemento x , em primeiro lugar teríamos de encontrar x na lista $T[h(x.chave)]$ para podermos atualizar o próximo atributo do predecessor de x . Com listas simplesmente ligadas, a eliminação e a busca teriam os mesmos tempos de execução assintóticos.)

Análise do hash com encadeamento

Como é o desempenho do hashing com encadeamento? Em particular, quanto tempo ele leva para procurar um elemento com uma determinada chave?

Dada uma tabela de espalhamento T com m posições que armazena n elementos, definimos o **fator de carga** α para T como n/m , isto é, o número médio de elementos armazenados em uma cadeia. Nossa análise será em termos de α , que pode ser menor, igual ou maior que 1.

O comportamento do pior caso do hashing com encadeamento é terrível: todas as n chaves vão para a mesma posição após o hashing, criando uma lista de comprimento n . Portanto, o tempo do pior caso para a busca é (n) mais o tempo necessário para calcular a função hash — não é melhor do que seria se usássemos uma única lista ligada para todos os elementos. É claro que não usamos as tabelas de espalhamento por seu desempenho no pior caso. (Todavia, o hashing perfeito, descrito na Seção 11.5, realmente oferece bom desempenho no pior caso quando o conjunto de chaves é estático.)

O desempenho do hashing para o caso médio depende de como a função hash h distribui o conjunto de chaves a armazenar entre as m posições, em média. A Seção 11.3 discute essas questões, mas, por enquanto, devemos considerar que qualquer elemento dado tem igual probabilidade de passar para qualquer das m posições após o hashing, independentemente do lugar para onde qualquer outro elemento tenha passado após essa operação. Denominamos essa premissa **hashing uniforme simples**.

Para $j = 0, 1, \dots, m - 1$, vamos denotar o comprimento da lista $T[j]$ por n_j , de modo que

$$n = n_0 + n_1 + \dots + n_{m-1}, \quad (11.1)$$

e o valor esperado de n_j é $E[n_j] = \alpha = n/m$.

Supomos que o tempo $O(1)$ é suficiente para calcular o valor hash $h(k)$, de modo que o tempo requerido para procurar um elemento com chave k depende linearmente do comprimento $n_{h(k)}$ da lista $T[h(k)]$. Deixando de lado o tempo $O(1)$ necessário para calcular a função hash e acessar a posição $h(k)$, vamos considerar o número esperado de elementos examinados pelo algoritmo de busca, isto é, o número de elementos na lista $T[h(k)]$ que o algoritmo verifica para ver se qualquer deles tem uma chave igual a k . Consideraremos dois casos. No primeiro, a busca não é bem-sucedida: nenhum elemento na tabela tem a chave k . No segundo caso, a busca consegue encontrar um elemento com chave k .

Teorema 11.1

Em uma tabela de espalhamento na qual as colisões são resolvidas por encadeamento, uma busca mal sucedida demora o tempo do caso médio $(1 + \alpha)$, sob a hipótese de hashing uniforme simples.

Prova Sob a hipótese de hashing uniforme simples, qualquer chave k ainda não armazenada na tabela tem igual probabilidade de ocupar qualquer das m posições após o hash. O tempo esperado para procurar sem sucesso uma chave k é o tempo esperado para pesquisar até o fim da lista $T[h(k)]$, que tem o comprimento esperado $E[n_{h(k)}] = \alpha$. Assim, o número esperado de elementos examinados em uma busca mal sucedida é α , e o tempo total exigido (incluindo o tempo para se calcular $h(k)$) é $(1 + \alpha)$.

A situação para uma busca bem-sucedida é ligeiramente diferente, já que a probabilidade de cada lista ser pesquisada não é a mesma. Em vez disso, a probabilidade de uma lista ser pesquisada é proporcional ao número de elementos que ela contém. Todavia, o tempo de busca esperado ainda é $(1 + \alpha)$.

Teorema 11.2

Em uma tabela de espalhamento na qual as colisões são resolvidas por encadeamento, uma busca bem-sucedida demora o tempo do caso médio $(1 + \alpha)$ se considerarmos hashing uniforme simples.

Prova Supomos que o elemento que está sendo pesquisado tem igual probabilidade de ser qualquer dos n elementos armazenados na tabela. O número de elementos examinados durante uma busca bem-sucedida para um elemento x é uma unidade maior que o número de elementos que aparecem antes de x na lista de x . Como elementos novos são colocados à frente na lista, os elementos antes de x na lista foram todos inseridos após x ser inserido. Para determinar o número esperado de elementos examinados, tomamos a média, sobre os n elementos x na tabela, de 1 mais o número esperado de elementos adicionados à lista de x depois que x foi adicionado à lista. Seja x_i o i -ésimo elemento inserido na tabela, para $i = 1, 2, \dots, n$, e seja $k_i = x_i.chave$. Para chaves k_i e k_j , definimos a variável aleatória indicadora $X_{ij} = I\{h(k_i) = h(k_j)\}$. Sob a hipótese de hashing uniforme simples, temos $\Pr\{h(k_i) = h(k_j)\} = 1/m$ e, então, pelo Lema 5.1, $E[X_{ij}] = 1/m$. Assim, o número esperado de elementos examinados em uma busca bem-sucedida é

$$\begin{aligned} E\left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij}\right)\right] \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}]\right) \quad \text{por linearidade da esperança} \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m}\right) \\ &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i\right) \\ &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2}\right) \quad \text{pela equação (A.1))} \\ &= 1 + \frac{n-1}{2m} \\ &= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \end{aligned}$$

Assim, o tempo total exigido para uma busca bem-sucedida (incluindo o tempo para calcular a função hash) é $(2 + \alpha/2 - \alpha/2n) = (1 + \alpha)$.

O que significa essa análise? Se o número de posições da tabela de espalhamento é no mínimo proporcional ao número de elementos na tabela, temos $n = O(m)$ e, conseqüentemente, $\alpha = n/m = O(m)/m = O(1)$. Assim, a busca

demora tempo constante na média. Visto que a inserção demora o tempo $O(1)$ no pior caso e a eliminação demora o tempo $O(1)$ no pior caso quando as listas são duplamente ligadas, podemos suportar todas as operações de dicionário no tempo $O(1)$ na média.

Exercícios

- 11.2-1** Suponha que utilizamos uma função hash h para efetuar o hashing de n chaves distintas em um arranjo T de comprimento m . Considerando hashing uniforme simples, qual é o número esperado de colisões? Mais precisamente, qual é a cardinalidade esperada de $\{\{k, l\} : k \neq l \text{ e } h(k) = h(l)\}$?
- 11.2-2** Demonstre o que acontece quando inserimos as chaves 5, 28, 19, 15, 20, 33, 12, 17, 10 em uma tabela de espalhamento com colisões resolvidas por encadeamento. Considere uma tabela com nove posições e a função hash $h(k) = k \bmod 9$.
- 11.2-3** O professor Marley apresenta a hipótese de que podemos obter ganhos substanciais de desempenho modificando o esquema de encadeamento para manter cada lista em sequência ordenada. Como a modificação do professor afeta o tempo de execução para pesquisas bem-sucedidas, mal sucedidas, inserções e eliminações?
- 11.2-4** Sugira como alocar e desalocar armazenamento para elementos dentro da própria tabela de espalhamento ligando todas as posições não utilizadas em uma lista livre. Suponha que uma posição pode armazenar um sinalizador e um elemento mais um ponteiro ou dois ponteiros. Todas as operações de dicionário e de lista livre devem ser executadas no tempo esperado $O(1)$. A lista livre precisa ser duplamente ligada ou uma lista livre simplesmente ligada é suficiente?
- 11.2-5** Suponha que estejamos armazenando um conjunto de n chaves em uma tabela de espalhamento de tamanho m . Mostre que, se as chaves forem extraídas de um universo U com $|U| > nm$, então U tem um subconjunto de tamanho n composto por chaves que passam todas para uma mesma posição após o hashing, de modo que o tempo de busca do pior caso para o hashing com encadeamento é (n) .
- 11.2-6** Suponha que armazenemos n chaves em uma tabela de espalhamento de tamanho m , com colisões resolvidas por encadeamento e que conheçamos o comprimento de cada cadeia, incluindo o comprimento L da cadeia mais longa. Descreva um procedimento que seleciona uma chave uniformemente ao acaso entre as chaves na tabela de espalhamento e a retorna no tempo esperado $O(L \cdot (1 + 1/\alpha))$.

11.3 FUNÇÕES HASH

Nesta seção, discutiremos algumas questões relacionadas ao projeto de boas funções hash e depois apresentaremos três esquemas para sua criação. Dois dos esquemas, hash por divisão e hash por multiplicação, são heurísticos por natureza, enquanto o terceiro esquema, hash universal, utiliza a aleatorização para oferecer um desempenho que podemos provar que é bom.

O que faz uma boa função hash ?

Uma boa função hash satisfaz (aproximadamente) a premissa do hashing uniforme simples: cada chave tem igual probabilidade de passar para qualquer das m posições por uma operação de hash, independentemente da posição que qualquer outra chave ocupou após o hash. Infelizmente, normalmente não temos nenhum meio de verificar essa condição, já que raramente conhecemos a distribuição de probabilidade da qual as chaves são extraídas. Além disso, as

chaves poderiam não ser extraídas independentemente. Ocasionalmente conhecemos a distribuição. Por exemplo, se soubermos que as chaves são números reais aleatórios k , independente e uniformemente distribuídos na faixa $0 \leq k < 1$, então a função hash

$$h(k) = km$$

satisfaz a condição de hashing uniforme simples.

Na prática, muitas vezes, podemos usar técnicas heurísticas para criar uma função hash que funciona bem. Informações qualitativas sobre a distribuição de chaves podem ser úteis nesse processo de projeto. Por exemplo, considere a tabela de símbolos de um compilador, na qual as chaves são cadeias de caracteres que representam identificadores em um programa. Símbolos estreitamente relacionados, como `pt` e `pts`, frequentemente ocorrem no mesmo programa. Uma boa função hash minimizaria a chance de tais variações passarem para a mesma posição após o hashing.

Uma boa abordagem deriva o valor hash de um modo que esperamos seja independente de quaisquer padrões que possam existir nos dados. Por exemplo, o “método de divisão” (discutido na Seção 11.3.1) calcula o valor hash como o resto quando a chave é dividida por um número primo especificado. Esse método frequentemente dá bons resultados, desde que escolhamos um número primo que não esteja relacionado com quaisquer padrões na distribuição de chaves.

Finalmente, observamos que algumas aplicações de funções hash poderiam exigir propriedades mais fortes que as oferecidas pelo hashing uniforme simples. Por exemplo, poderíamos querer que chaves que de certo modo são mais “próximas” derivem valores hash muito afastados um do outro. (Essa propriedade é especialmente desejável quando estamos usando sondagem linear, definida na Seção 11.4.) O hash universal, descrito na Seção 11.3.3, muitas vezes, fornece as propriedades desejadas.

Interpretação de chaves como números naturais

A maior parte das funções hash considera como universo de chaves o conjunto $\Sigma^* = \{0, 1, 2, \dots\}$ de números naturais. Assim, se as chaves não forem números naturais, temos de encontrar um modo de *interpretá-las* como números naturais. Por exemplo, podemos interpretar uma cadeia de caracteres como um inteiro expresso em uma notação de raiz adequada. Assim, poderíamos interpretar o identificador `pt` como o par de inteiros decimais (112, 116), já que `p` = 112 e `t` = 116 no conjunto de caracteres ASCII; então, expresso como um inteiro de raiz 128, `pt` se torna $(112 \cdot 128) + 116 = 14.452$. No contexto de uma determinada aplicação, normalmente podemos elaborar algum método para interpretar cada chave como um número natural (possivelmente grande). No que vem a seguir, supomos que as chaves são números naturais.

11.3.1 O MÉTODO DE DIVISÃO

No *método de divisão* para criar funções hash, mapeamos uma chave k para uma de m posições, tomando o resto da divisão de k por m . Isto é, a função hash é

$$h(k) = k \bmod m.$$

Por exemplo, se a tabela de espalhamento tem tamanho $m = 12$ e a chave é $k = 100$, então $h(k) = 4$. Visto que exige uma única operação de divisão, o hash por divisão é bastante rápido.

Quando utilizamos o método de divisão, em geral evitamos certos valores de m . Por exemplo, m não deve ser uma potência de 2, já que, se $m = 2^p$, então $h(k)$ será somente o grupo de p bits de ordem mais baixa de k . A menos que saibamos que todos os padrões de p bits de ordem baixa são igualmente prováveis, é melhor garantir que em nosso projeto a função hash dependa de todos os bits da chave. Como o Exercício 11.3-3 lhe pede para mostrar, escolher $m = 2^p - 1$ quando k é uma cadeia de caracteres interpretada em raiz 2^p pode ser uma escolha ruim porque permutar os caracteres de k não altera seu valor hash.

Muitas vezes, um primo não muito próximo de uma potência exata de 2 é uma boa escolha para m . Por exemplo, suponha que desejemos alocar uma tabela de espalhamento, com colisões resolvidas por encadeamento, para conter aproximadamente $n = 2.000$ cadeias de caracteres, onde um caractere tem oito bits. Não nos importamos de examinar uma média de três elementos em uma busca mal sucedida e, assim, alocamos uma tabela de espalhamento de tamanho $m \geq 701$. Podemos escolher o número 701 porque é um primo próximo de $2.000/3$, mas não próximo de nenhuma potência de 2. Tratando cada chave k como um inteiro, nossa função hash seria

$$h(k) = k \bmod 701 .$$

11.3.2 O MÉTODO DE MULTIPLICAÇÃO

O *método de multiplicação* para criar funções hash funciona em duas etapas. Primeiro, multiplicamos a chave k por uma constante A na faixa $0 < A < 1$ e extraímos a parte fracionária de kA . Em seguida, multiplicamos esse valor por m e tomamos o piso do resultado. Resumindo, a função hash é

$$h(k) = \lfloor m(kA \bmod 1) \rfloor ,$$

onde “ $kA \bmod 1$ ” significa a parte fracionária de kA , isto é, $kA - \lfloor kA \rfloor$.

Uma vantagem do método de multiplicação é que o valor de m não é crítico. Em geral, nós o escolhemos de modo a ser uma potência de 2 ($m = 2^p$ para algum inteiro p) já que podemos implementar facilmente a função na maioria dos computadores do modo descrito a seguir. Suponha que o tamanho da palavra da máquina seja w bits e que k caiba em uma única palavra. Restringimos A a ser uma fração da forma $s/2^w$, onde s é um inteiro na faixa $0 < s < 2^w$. Referindo-nos à Figura 11.4, primeiro multiplicamos k pelo inteiro $s = A \cdot 2^w$ de w bits. O resultado é um valor de $2w$ bits $r_1 2^w + r_0$, onde r_1 é a palavra de ordem alta do produto e r_0 é a palavra de ordem baixa do produto. O valor hash de p bits desejado consiste nos p bits mais significativos de r_0 .

Embora esse método funcione com qualquer valor da constante A , funciona melhor com alguns valores que com outros. A escolha ótima depende das características dos dados aos quais o hash está sendo aplicado. Knuth [185] sugere que

$$A \approx (\sqrt{5} - 1) / 2 = 0,6180339887... \quad (11.2)$$

tem boa possibilidade de funcionar razoavelmente bem.

Como exemplo, suponha que tenhamos $k = 123456$, $p = 14$, $m = 2^{14} = 16384$ e $w = 32$. Adaptando a sugestão de Knuth, escolhemos A como a fração da forma $s/2^{32}$ mais próxima a $(\sqrt{5}-1)/2$, isto é, $A = 2654435769/2^{32}$. Então, $k \times s = 327706022297664 = (76300 \cdot 2^{32}) + 17612864$ e, assim, $r_1 = 76300$ e $r_0 = 17612864$. Os 14 bits mais significativos de r_0 formam o valor $h(k) = 67$.

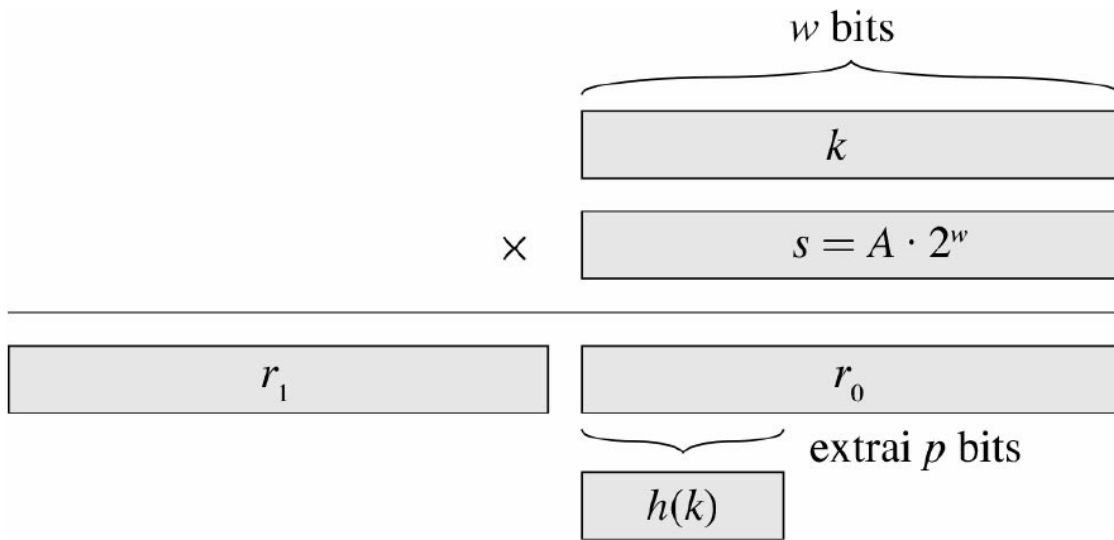


Figura 11.4 O método de multiplicação para hash. A representação de w bits da chave k é multiplicada pelo valor de w bits $s = A \times 2^w$, onde $0 < A < 1$ é uma constante adequada. Os p bits de ordem mais alta da metade inferior de w bits do produto formam o valor hash desejado $h(k)$.

11.3.3 ★ HASHING UNIVERSAL

Se um adversário malicioso escolher as chaves às quais o hashing deverá ser aplicado por alguma função hash fixa, ele pode escolherá n chaves que passem para a mesma posição após o hash, o que resulta em um tempo médio de recuperação igual a $Q(n)$. Qualquer função hash fixa é vulnerável a esse terrível comportamento do pior caso; a única maneira eficaz de melhorar a situação é escolher a função hash *aleatoriamente*, de um modo que seja *independente* das chaves que realmente serão armazenadas. Essa abordagem, denominada **hashing universal**, pode resultar em um desempenho demonstravelmente bom na média, não importando as chaves escolhidas pelo adversário.

No hashing universal, no início da execução selecionamos a função hash aleatoriamente de uma classe de funções cuidadosamente projetada. Como no caso do quicksort, a aleatorização garante que nenhuma entrada isolada evocará sempre o comportamento do pior caso. Como selecionamos a função hash aleatoriamente, o algoritmo poderá se comportar de modo diferente em cada execução ainda que a entrada seja a mesma, garantindo um bom desempenho do caso médio para qualquer entrada. Retornando ao exemplo da tabela de símbolos de um compilador, verificamos que agora a escolha de identificadores pelo programador não pode provocar consistentemente um desempenho ruim do hash. O desempenho ruim ocorre apenas quando o compilador escolhe uma função hash aleatória que faz com que o resultado do hash aplicado ao conjunto de identificadores seja ruim, mas a probabilidade de ocorrer essa situação é pequena, e é a mesma para qualquer conjunto de identificadores do mesmo tamanho.

Seja uma coleção finita de funções hash que mapeiam um dado universo U de chaves para a faixa $\{0, 1, \dots, m - 1\}$. Dizemos que ela é **universal** se, para cada par de chaves distintas $k, l \in U$, o número de funções hash $h \in$ para as quais $h(k) = h(l)$ é no máximo $|U|/m$. Em outras palavras, com uma função hash escolhida aleatoriamente de , a chance de uma colisão entre chaves distintas k e l não é maior que a chance $1/m$ de uma colisão se $h(k)$ e $h(l)$ fossem escolhidas aleatória e independentemente do conjunto $\{0, 1, \dots, m - 1\}$.

O teorema a seguir, mostra que uma classe universal de funções hash oferece bom comportamento do caso médio. Lembre-se de que n_i denota o comprimento da lista $T[i]$.

Teorema 11.3

Suponha que uma função hash h seja escolhida aleatoriamente de uma coleção universal de funções hash e usada para aplicar hash às n chaves em uma tabela T de tamanho m , usando encadeamento para resolver colisões. Se a chave k

não estiver na tabela, o comprimento esperado $E[n_h(k)]$ da lista para a qual a chave k passa após aplicação do hash é no máximo o fator de carga $\alpha = n/m$. Se a chave k estiver na tabela, o comprimento esperado $E[n_h(k)]$ da lista que contém a chave k é no máximo $1 + \alpha$.

Prova Notamos que, aqui, as esperanças referem-se à escolha da função hash e não dependem de quaisquer premissas adotadas para a distribuição das chaves. Para cada par k e l de chaves distintas, defina a variável aleatória indicadora $X_{kl} = I\{h(k) = h(l)\}$. Visto que, pela definição de uma coleção de funções hash, um único par de chaves colide com probabilidade de no máximo $1/m$, temos $\Pr\{h(k) = h(l)\} \leq 1/m$. Portanto, pelo Lema 5.1, temos $E[X_{kl}] \leq 1/m$.

Em seguida, definimos para cada chave k a variável aleatória Y_k , que é igual ao número de chaves diferentes de k que passam para a mesma posição de k após a aplicação do hash, de modo que

$$Y_k = \sum_{\substack{l \in T \\ l \neq k}} X_{kl}.$$

Portanto, temos

$$\begin{aligned} E[Y_k] &= E\left[\sum_{\substack{l \in T \\ l \neq k}} X_{kl}\right] \\ &= \sum_{\substack{l \in T \\ l \neq k}} E[X_{kl}] \quad (\text{por linearidade da esperança}) \\ &\leq \sum_{\substack{l \in T \\ l \neq k}} \frac{1}{m} \end{aligned}$$

O restante da prova depende de a chave k estar ou não na tabela T .

- Se $k \notin T$, então $n_h(k) = Y_k$ e $|\{l : l \in T \text{ e } l \neq k\}| = n$. Assim, $E n_h(k) = E Y_k \leq n/m = \alpha$.
- Se $k \in T$, então, como a chave k aparece na lista $Th(k)$ e a contagem Y_k não inclui a chave k , temos $n_h(k) = Y_k + 1$ e $|\{l : l \in T \text{ e } l \neq k\}| = n - 1$. Assim, $E[n_h(k)] = E[Y_k] \leq (n - 1)/m + 1 = 1 + \alpha - 1/m < 1 + \alpha$.

O corolário a seguir, diz que o hash universal nos dá a compensação desejada: agora ficou impossível um adversário escolher uma sequência de operações que force o tempo de execução do pior caso. Com a aleatorização inteligente da escolha da função hash em tempo de execução, garantimos que podemos processar toda sequência de operações com um bom tempo de execução do caso médio.

Corolário 11.4

Usando hash universal e resolução de colisão por encadeamento em uma tabela inicialmente vazia com m posições, tratar qualquer sequência de n operações INSERT, SEARCH e DELETE contendo $O(m)$ operações INSERT demora o tempo esperado (n) .

Prova Como o número de inserções é $O(m)$, temos $n = O(m)$ e, assim, $\alpha = O(1)$. As operações INSERT e DELETE demoram tempo constante e, pelo Teorema 11.3, o tempo esperado para cada operação SEARCH é $O(1)$. Assim, por linearidade da esperança, o tempo esperado para a sequência de operações inteira é $O(n)$. Visto que cada operação leva o tempo (1) , decorre o limite (n) .

Projeto de uma classe universal de funções hash

É bastante fácil projetar uma classe universal de funções hash, como um pouco de teoria dos números nos ajudará a demonstrar. Se você não estiver familiarizado com a teoria dos números, será bom consultar o Capítulo 31 antes.

Começamos escolhendo um número primo p suficientemente grande para que toda chave k possível esteja no intervalo 0 a $p - 1$, inclusive. Seja $\mathbb{Z}_p$ o conjunto $\{0, 1, \dots, p - 1\}$ e seja $\mathbb{Z}_p^*$ o conjunto $\{1, 2, \dots, p - 1\}$. Visto que p é primo, podemos resolver equações de módulo p com os métodos dados no Capítulo 31. Como supomos que o tamanho do universo de chaves é maior que o número de posições na tabela de espalhamento, temos $p > m$.

Agora definimos a função hash h_{ab} para qualquer $a \in \mathbb{Z}_p^*$ e qualquer $b \in \mathbb{Z}_p$ usando uma transformação linear seguida por reduções de módulo p e, então, de módulo m :

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m. \quad (11.3)$$

Por exemplo, com $p = 17$ e $m = 6$, temos $h_{3,4}(8) = 5$. A família de todas essas funções hash é

$$\mathcal{H}_{pm} = \{h_{ab} : a \in \mathbb{Z}_p^* \text{ e } b \in \mathbb{Z}_p\}. \quad (11.4)$$

Cada função hash $h_{a,b}$ mapeia p para m . Essa classe de funções hash tem a interessante propriedade de que o tamanho m da faixa de saída é arbitrário — não necessariamente primo —, uma característica que usaremos na Seção 11.5. Visto que temos $p - 1$ escolhas para a e que há p escolhas para b , a coleção $\mathcal{H}_{pm}$ contém $p(p - 1)$ funções hash.

Teorema 11.5

A classe $\mathcal{H}_{pm}$ de funções hash definida pelas equações (11.3) e (11.4) é universal.

Prova Considere duas chaves distintas k e l de $\mathbb{Z}_p$, de modo que $k \neq l$. Para uma dada função hash h_{ab} fazemos

$$r = (ak + b) \bmod p,$$

$$s = (al + b) \bmod p.$$

Primeiro observamos que $r \neq s$. Por quê? Observe que

$$r - s \equiv a(k - l) \pmod{p}.$$

Decorre que $r \neq s$ porque p é primo e ambos, a e $(k - l)$, não são zero módulo p e, assim, seu produto também deve ser não zero módulo p pelo Teorema 31.6. Portanto, na computação de qualquer h_{ab} em $\mathcal{H}_{pm}$, entradas distintas k e l mapeiam para valores distintos r e s módulo p ; ainda não há nenhuma colisão no “nível mod p ”. Além disso, cada uma das $p(p - 1)$ escolhas possíveis para o par (a, b) com $a \neq 0$ produz um par resultante (r, s) diferente com $r \neq s$, já que podemos resolver para a e b dados r e s :

$$a = ((r - s)((k - l)^{-1} \bmod p)) \bmod p,$$

$$b = (r - ak) \bmod p,$$

onde $((k - l)^{-1} \bmod p)$ denota o inverso multiplicativo único, módulo p , de $k - l$. Como existem apenas $p(p - 1)$ pares (r, s) possíveis com $r \neq s$, há uma correspondência de um para um entre pares (a, b) com $a \neq 0$ e pares (r, s) com $r \neq s$. Assim, para qualquer par de entradas k e l dado, se escolhermos (a, b) uniformemente ao acaso de $\mathcal{H}_{pm}$, o par resultante (r, s) terá igual probabilidade de ser qualquer par de valores distintos módulo p .

Então, a probabilidade de chaves distintas k e l colidirem é igual à probabilidade de $r \equiv s \pmod{m}$ quando r e s são escolhidos aleatoriamente como valores distintos módulo p . Para um dado valor de r , dos $p - 1$ valores restantes possíveis para s , o número de valores s tais que $s \neq r$ e $s \equiv r \pmod{m}$ é no máximo

$$\begin{aligned}\lceil p/m \rceil - 1 &\leq ((p + m - 1)/m) - 1 && \text{(pela desigualdade (3.6))} \\ &= (p - 1)/m.\end{aligned}$$

A probabilidade de s colidir com r quando reduzido módulo m é no máximo $((p - 1)/m)/(p - 1) = 1/m$.

Assim, para qualquer par de valores distintos $k, l \in p$,

$$\Pr\{h_{ab}(k) = h_{ab}(l)\} \leq 1/m,$$

de modo que p_m é, de fato, universal.

Exercícios

- 11.3-1** Suponha que desejemos buscar em uma lista ligada de comprimento n , onde cada elemento contém um chave k juntamente com um valor hash $h(k)$. Cada chave é uma cadeia de caracteres longa. Como poderíamos tirar proveito dos valores hash ao procurar na lista um elemento com uma chave específica?
- 11.3-2** Suponha que aplicamos hash a uma cadeia de r caracteres em m posições, tratando-a como um número de raiz 128, depois usando o método de divisão. Podemos representar facilmente o número m como uma palavra de computador de 32 bits, mas a cadeia de r caracteres, tratada como um número de raiz 128, ocupa muitas palavras. Como podemos aplicar o método de divisão para calcular o valor hash da cadeia de caracteres sem usar mais do que um número constante de palavras de armazenamento fora da própria cadeia?
- 11.3-3** Considere uma versão do método de divisão, na qual $h(k) = k \bmod m$, onde $m = 2^p - 1$ e k é uma cadeia de caracteres interpretada em raiz 2^p . Mostre que, se pudermos derivar a cadeia x da cadeia y por permutação de seus caracteres, o hash aplicado a x e y resultará no mesmo valor para ambos. Dê exemplo de uma aplicação na qual essa propriedade seria indesejável em uma função hash.
- 11.3-4** Considere uma tabela de espalhamento de tamanho $m = 1.000$ e uma função hash correspondente $h(k)$ igual a $m(kA \bmod 1)$ para $A = (\sqrt{5}-1)/2$. Calcule as localizações para as quais são mapeadas as chaves 61, 62, 63, 64 e 65.
- 11.3-5** ★ Defina uma família de funções hash de um conjunto finito U para um conjunto finito B como *e-universal* se, para todos os pares de elementos distintos k e l em U ,

$$\Pr\{h(k) = h(l)\} \leq \epsilon,$$

onde a probabilidade refere-se à escolha aleatória da função hash h na família H . Mostre que uma família *e-universal* de funções hash deve ter

$$\epsilon \geq \frac{1}{|B|} - \frac{1}{U}$$

- 11.3-6** ★ Seja U o conjunto de ênuclas de valores extraídos de p , e seja $B = p$, onde p é primo. Defina a função hash $h_b : U \rightarrow B$ para $b \in p$ para um ênucla de entrada $\langle a_0, a_1, \dots, a_{n-1} \rangle$ extraída de U por

$$h_b\left(\langle a_0, a_1, \dots, a_{n-1} \rangle\right) = \left(\sum_{j=0}^{n-1} a_j b^j\right) \bmod p,$$

e seja $= \{h_b : b \in p.\}$. Demonstre que é $((n-1)/p)$ -universal, de acordo com a definição de $\in$ -universal no Exercício 11.3-5. (Sugestão: Veja o Exercício 31.4-4.)

11.4 ENDEREÇAMENTO ABERTO

Em *endereçamento aberto*, todos os elementos ficam na própria tabela de espalhamento. Isto é, cada entrada da tabela contém um elemento do conjunto dinâmico ou `NIL`. Ao procurar um elemento, examinamos sistematicamente as posições da tabela até encontrar o elemento desejado ou até confirmar que o elemento não está na tabela. Diferentemente do encadeamento, não existe nenhuma lista e nenhum elemento armazenado fora da tabela. Assim, no endereçamento aberto, a tabela de espalhamento pode “ficar cheia”, de tal forma que nenhuma inserção adicional pode ser feita; o fator de carga α nunca pode exceder 1.

É claro que poderíamos armazenar as listas ligadas para encadeamento no interior da tabela de espalhamento, nas posições não utilizadas de outro modo (veja o Exercício 11.2-4), mas a vantagem do endereçamento aberto é que ele evita por completo os ponteiros. Em vez de seguir ponteiros, *calculamos* a sequência de posições a examinar. A memória extra liberada por não armazenarmos ponteiros fornece à tabela de espalhamento um número maior de posições para a mesma quantidade de memória, o que produz potencialmente menor número de colisões e recuperação mais rápida.

Para executar inserção usando endereçamento aberto, examinamos sucessivamente, ou *sondamos*, a tabela de espalhamento até encontrar uma posição vazia na qual inserir a chave. Em vez de ser fixa na ordem $0, 1, \dots, m-1$ (o que exige o tempo de busca $Q(n)$), a sequência de posições sondadas *depende da chave que está sendo inserida*. Para determinar quais serão as posições a sondar, estendemos a função hash para incluir o número da sondagem (a partir de 0) como uma segunda entrada. Assim, a função hash se torna

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}.$$

Com endereçamento aberto, exigimos que, para toda chave k , a *sequência de sondagem*

$$\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$$

seja uma permutação de $\langle 0, 1, \dots, m-1 \rangle$, de modo que toda posição da tabela de espalhamento seja eventualmente considerada uma posição para uma nova chave, à medida que a tabela é preenchida. No pseudocódigo a seguir, supomos que os elementos na tabela de espalhamento T são chaves sem informações satélites; a chave k é idêntica ao elemento que contém a chave k . Cada posição contém uma chave ou `NIL` (se a posição estiver vazia). O procedimento `HASH-SEARCH` tem como entrada uma tabela de espalhamento T e uma chave k . Devolve o número da posição onde armazena chave k ou sinaliza um erro porque a tabela de espalhamento já está cheia.

`HASH-INSERT( $T, k$ )`

1 $i = 0$

2 **repeat** $j = h(k, i)$

3 **if** $T[j] == \text{NIL}$

4 $T[j] = k$

5 **return** j

6 **else** $i = i + 1$

7 **until** $i == m$

8 **error** “estouro da tabela”

O algoritmo que procura a chave k sonda a mesma sequência de posições que o algoritmo de inserção examinou quando a chave k foi inserida. Portanto, a busca pode terminar (sem sucesso) quando encontra uma posição vazia, já que k teria sido inserido ali e não mais adiante em sua sequência de sondagem. (Esse argumento supõe que não há eliminação de chaves na tabela de espalhamento.) O procedimento `HASH-SEARCH` tem como entrada uma tabela de espalhamento T e um chave k , e devolve j se verificar que a posição j contém a chave k , ou `NIL` se a chave k não estiver presente na tabela T .

```

HASH-SEARCH( $T, k$ )
1  $i = 0$ 
2 repeat
3    $j = h(k, i)$ 
4   if  $T[j] == k$ 
5     return  $j$ 
6    $i = i + 1$ 
7 until  $T[j] == \text{NIL}$  ou  $i == m$ 
8 return NIL

```

Eliminar algo em uma tabela de espalhamento de endereço aberto é difícil. Quando eliminamos uma chave da posição i , não podemos simplesmente marcar essa posição como vazia, nela armazenando `NIL`. Se fizéssemos isso, poderíamos não conseguir recuperar nenhuma chave k em cuja inserção tivéssemos sondado a posição i e verificado que ela estava ocupada. Podemos resolver esse problema marcando a posição nela armazenando o valor especial `DELETED` em vez de `NIL`. Então, modificaríamos o procedimento `HASH-INSERT` para tratar tal posição como se ela estivesse vazia, de modo a podermos inserir uma nova chave. Não é necessário modificar `HASH-SEARCH`, já que ele passará por valores `DELETED` enquanto estiver pesquisando. Entretanto, quando usamos o valor especial `DELETED`, os tempos de busca não dependem mais do fator de carga α . Por essa razão, o encadeamento é mais comumente selecionado como uma técnica de resolução de colisões quando precisamos eliminar chaves.

Em nossa análise, consideramos *hash uniforme*: a sequência de sondagem de cada chave tem igual probabilidade de ser qualquer uma das $m!$ permutações de $\langle 0, 1, \dots, m-1 \rangle$. O hash uniforme generaliza a noção de hash uniforme simples definida anteriormente para uma função hash que produz não apenas um número único, mas toda uma sequência de sondagem. Contudo, o verdadeiro hash uniforme é difícil de implementar e, na prática, são usadas aproximações adequadas (como o hash duplo, definido a seguir).

Examinaremos três técnicas comumente utilizadas para calcular as sequências de sondagem exigidas para o endereçamento aberto: sondagem linear, sondagem quadrática e hash duplo. Todas essas técnicas garantem que $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ é uma permutação de $\langle 0, 1, \dots, m-1 \rangle$ para cada chave k . Porém, nenhuma delas cumpre o requisito do hash uniforme, já que nenhuma consegue gerar mais de m_2 sequências de sondagem diferentes (em vez das $m!$ que o hash uniforme exige). O hash duplo tem o maior número de sequências de sondagem e, como seria de esperar, parece dar os melhores resultados.

Sondagem linear

Dada uma função hash comum $h': U \rightarrow \{0, 1, \dots, m-1\}$, à qual nos referimos como uma *função hash auxiliar*, o método de *sondagem linear* usa a função hash

$$h(k, i) = (h'(k) + i) \bmod m$$

para $i = 0, 1, \dots, m-1$. Dada a chave k , primeiro sondamos $T[h'(k)]$, isto é, a posição dada pela função hash auxiliar. Em seguida, sondamos a posição $T[h'(k) + 1]$, e assim por diante até a posição $T[m-1]$. Depois, voltamos às

posições $T[0]$, $T[1]$, ..., até finalmente sondarmos a posição $T[h'(k) - 1]$. Como a sondagem inicial determina toda a sequência de sondagem, há somente m sequências de sondagem distintas.

A sondagem linear é fácil de implementar, mas sofre de um problema conhecido como **agrupamento primário**. Longas sequências de posições ocupadas se acumulam, aumentando o tempo médio de busca. Os agrupamentos surgem porque uma posição vazia precedida por i posições cheias é preenchida em seguida com probabilidade $(i + 1)/m$. Longas sequências de posições ocupadas tendem a ficar mais longas, e o tempo médio de busca aumenta.

Sondagem quadrática

A **sondagem quadrática** utiliza uma função hash da forma

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m, \quad (11.5)$$

onde h' é uma função hash auxiliar, c_1 e c_2 são constantes positivas auxiliares e $i = 0, 1, \dots, m - 1$. A posição inicial sondada é $T[h'(k)]$; posições posteriores sondadas são deslocadas por quantidades que dependem de modo quadrático do número da sondagem i . Esse método funciona muito melhor que a sondagem linear mas, para fazer pleno uso da tabela de espalhamento, os valores de c_1 , c_2 e m são restritos. O Problema 11-3 mostra um modo de selecionar esses parâmetros. Além disso, se duas chaves têm a mesma posição inicial de sondagem, então suas sequências de sondagem são iguais, já que $h(k_1, 0) = h(k_2, 0)$ implica $h(k_1, i) = h(k_2, i)$. Essa propriedade conduz a uma forma mais branda de agrupamento, denominada **agrupamento secundário**. Como na sondagem linear, a sondagem inicial determina a sequência inteira; assim, apenas m sequências de sondagem distintas são utilizadas.

Hash duplo

O hash duplo oferece um dos melhores métodos disponíveis para endereçamento aberto porque as permutações produzidas têm muitas das características de permutações escolhidas aleatoriamente. O **hash duplo** usa uma função hash da forma

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m,$$

onde h_1 e h_2 são funções hash auxiliares. A sondagem inicial vai à posição $T[h_1(k)]$; posições de sondagem sucessivas são deslocadas em relação às posições anteriores pela quantidade $h_2(k)$, módulo m . Assim, diferentemente do caso de sondagem linear ou quadrática, aqui a sequência de sondagem depende da chave k de duas maneiras, já que a posição inicial de sondagem, o deslocamento, ou ambos podem variar. A Figura 11.5 dá um exemplo de inserção por hash duplo.

O valor $h_2(k)$ e o tamanho m da tabela de espalhamento devem ser primos entre si para que a tabela de espalhamento inteira seja examinada (veja o Exercício 11.4-4). Uma forma conveniente de assegurar essa condição é que m seja uma potência de 2 e projetar h_2 de modo que sempre retorne um número ímpar. Outra maneira é que m seja primo e projetar h_2 de modo que sempre retorne um inteiro positivo menor que m . Por exemplo, poderíamos escolher m primo e fazer

$$\begin{aligned} h_1(k) &= k \bmod m, \\ h_2(k) &= 1 + (k \bmod m'), \end{aligned}$$

onde o valor de m escolhido é ligeiramente menor que m (digamos, $m - 1$). Por exemplo, se $k = 123456$, $m = 701$ e $m' = 700$, temos $h_1(k) = 80$ e $h_2(k) = 257$; assim, primeiro sondamos a posição 80 e depois examinamos cada 257ª posição (módulo m) até encontrarmos a chave ou termos examinado todas as posições.

Quando m é primo ou uma potência de 2, o hash duplo é melhor do que a sondagem linear ou quadrática no sentido de que são usadas (m_2) seqüências de sondagem em vez de (m) , já que cada par $(h_1(k), h_2(k))$ possível gera uma seqüência de sondagem distinta. O resultado é que, para tais valores de m , o desempenho do hash duplo parece ficar bem próximo do esquema “ideal” do hash uniforme.

Embora, em princípio, outros valores de m que não sejam primos nem potências de 2 poderiam ser utilizados com hash duplo, na prática torna-se mais difícil gerar $h_2(k)$ eficientemente de um modo que garanta que esse valor e m são primos entre si, em parte porque a densidade relativa $(m)/m$ de tais números pode ser pequena (veja equação (31.24)).

Análise de hash de endereço aberto

Como fizemos na análise do encadeamento, expressamos nossa análise do endereçamento aberto em termos do fator de carga $\alpha = n/m$ da tabela de espalhamento. É claro que no endereçamento aberto temos no máximo um elemento por posição e, portanto, $n \leq m$, o que implica $\alpha \leq 1$.

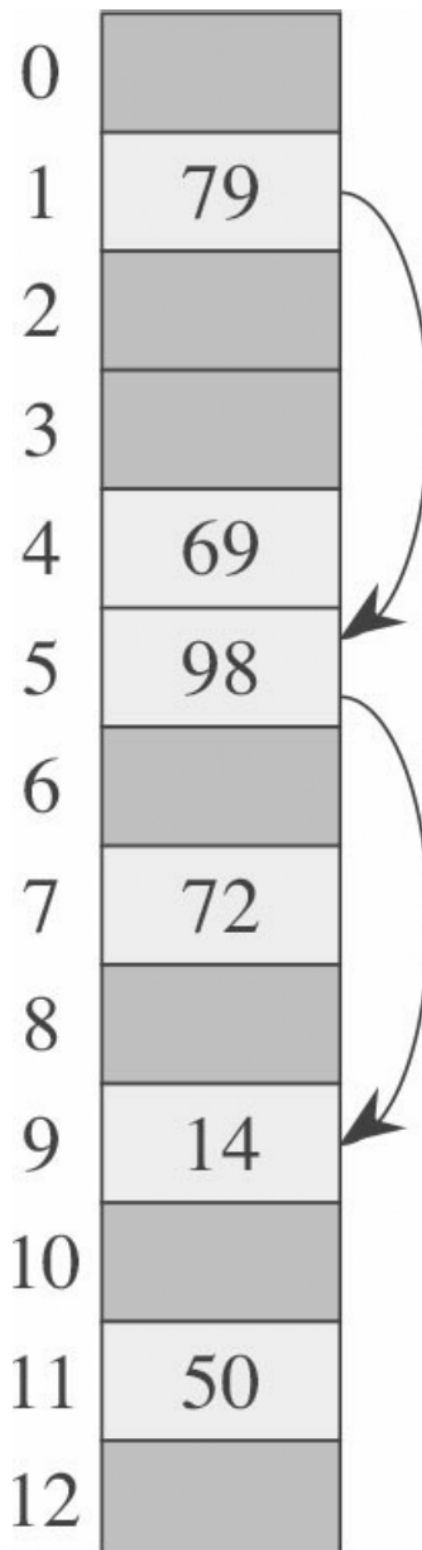


Figura 11.5 Inserção por hash duplo. Aqui temos uma tabela de espalhamento de tamanho 13 com $h_1(k) = k \bmod 13$ e $h_2(k) = 1 + (k \bmod 11)$. Como $14 \equiv 1 \pmod{13}$ e $14 \equiv 3 \pmod{11}$, inserimos a chave 14 na posição vazia 9, após examinar as posições 1 e 5 verificarmos que elas já estão ocupadas.

Supomos que estamos usando hash uniforme. Nesse esquema idealizado, a sequência de sondagem $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ usada para inserir ou procurar cada chave k tem igual probabilidade de ser qualquer permutação de

$\langle 0, 1, \dots, m-1 \rangle$. É claro que uma determinada chave tem uma sequência de sondagem fixa e única associada a ela; o que queremos dizer é que, considerando a distribuição de probabilidades no espaço de chaves e a operação da função hash nas chaves, cada sequência de sondagem possível é igualmente provável.

Agora, analisamos o número esperado de sondagens para hash com endereçamento aberto, adotando a premissa do hashing uniforme, começando com uma análise do número de sondagens realizadas em uma busca mal sucedida.

Teorema 11.6

Dada uma tabela de espalhamento de endereço aberto com fator de carga $\alpha = n/m < 1$, o número esperado de sondagens em uma busca mal sucedida é no máximo $1/(1 - \alpha)$, supondo hashing uniforme.

Prova Em uma busca mal sucedida, toda sondagem exceto a última acessa uma posição ocupada que não contém a chave desejada, e a última posição sondada está vazia. Vamos definir a variável aleatória X como o número de sondagens executadas em uma busca mal sucedida, e também definir o evento A_i , para $i = 1, 2, \dots$, como o evento em que ocorre uma i -ésima sondagem e ela é para uma posição ocupada. Então, o evento $\{X \geq i\}$ é a interseção dos eventos $A_1 \cap A_2 \cap \dots \cap A_{i-1}$. Limitaremos $\{X \geq i\}$ limitando $\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\}$. Pelo Exercício C.2-5,

$$\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\} = \frac{\Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \dots \Pr\{A_{i-1} \mid A_1 \cap A_2 \cap \dots \cap A_{i-2}\}}{\Pr\{A_{i-1} \mid A_1 \cap A_2 \cap \dots \cap A_{i-2}\}}.$$

Visto que há n elementos e m posições, $\Pr\{A_1\} = n/m$. Para $j > 1$, a probabilidade de existir uma j -ésima sondagem e ela ser para uma posição ocupada, dado que as primeiras $j-1$ sondagens foram para posições ocupadas, é $(n-j+1)/(m-j+1)$. Essa probabilidade decorre porque estaríamos encontrando um dos $(n-(j-1))$ elementos restantes em uma das $(m-(j-1))$ posições não examinadas e, pela premissa do hash uniforme, a probabilidade é a razão entre essas quantidades. Observando que $n < m$ implica $(n-j)/(m-j) \leq n/m$ para todo j tal que $0 \leq j < m$, temos para todo i tal que $1 \leq i \leq m$,

$$\begin{aligned} \Pr\{X \geq i\} &= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \dots \frac{n-i+2}{m-i+2} \\ &\leq \left(\frac{n}{m}\right)^{i-1} \\ &= \alpha^{i-1}. \end{aligned}$$

Agora, usamos a equação (C.25) para limitar o número esperado de sondagens:

$$\begin{aligned} E[X] &= \sum_{i=1}^{\infty} \Pr\{X \geq i\} \\ &\leq \sum_{i=1}^{\infty} \alpha^{i-1} \\ &= \sum_{i=0}^{\infty} \alpha^i \\ &= \frac{1}{1-\alpha} \end{aligned}$$

Esse limite de $1/(1-\alpha) = 1 + \alpha + \alpha^2 + \alpha^3 + \dots$ tem uma interpretação intuitiva. Sempre executamos a primeira sondagem. Com probabilidade de aproximadamente α , essa primeira sondagem encontra uma posição ocupada, de modo que precisamos de uma segunda sondagem. Com probabilidade de aproximadamente α^2 , as duas primeiras posições estão ocupadas, de modo que executamos uma terceira sondagem, e assim por diante.

Se α é uma constante, o Teorema 11.6 prevê que uma busca mal sucedida é executada no tempo $O(1)$. Por exemplo, se a tabela de espalhamento estiver cheia até a metade, o número médio de sondagens em uma busca mal sucedida é no máximo $1/(1 - 0,5) = 2$. Se ela estiver 90% cheia, o número médio de sondagens será no máximo $1/(1 - 0,9) = 10$.

O Teorema 11.6 nos dá o desempenho do procedimento `HASH-INSERT` quase imediatamente.

Corolário 11.7

Inserir um elemento em uma tabela de espalhamento de endereço aberto com fator de carga α exige no máximo $1/(1 - \alpha)$ sondagens, em média, considerando hash uniforme.

Prova Um elemento é inserido somente se houver espaço na tabela e, portanto, $\alpha < 1$. Inserir uma chave requer uma busca mal sucedida seguida pela colocação da chave na primeira posição vazia encontrada. Assim, o número esperado de sondagens é, no máximo, $1/(1 - \alpha)$.

Temos de trabalhar um pouco mais para calcular o número esperado de sondagens para uma busca bem-sucedida.

Teorema 11.8

Dada uma tabela de espalhamento de endereço aberto com fator de carga $\alpha < 1$, o número esperado de sondagens em uma busca bem sucedida é, no máximo, considerando hash uniforme e também que cada chave na tabela tem igual probabilidade de ser procurada.

Prova Uma busca de uma chave k reproduz a mesma sequência de sondagem que foi seguida na inserção do elemento com chave k . Pelo Corolário 11.7, se k foi a $(i + 1)$ -ésima chave inserida na tabela de espalhamento, o número esperado de sondagens efetuadas em uma busca de k é no máximo $1/(1 - i/m) = m/(m - i)$. O cálculo da média para todas as n chaves na tabela de espalhamento nos dá o número esperado de sondagens em uma busca bem-sucedida: Se a tabela de espalhamento estiver cheia até a metade, o número esperado de sondagens em uma busca bem-sucedida será menor que 1,387. Se a tabela de espalhamento estiver 90% cheia, o número esperado de sondagens será menor que 2,559.

Exercícios

- 11.4-1** Considere a inserção das chaves 10, 22, 31, 4, 15, 28, 17, 88, 59 em uma tabela de espalhamento de comprimento $m = 11$ usando endereçamento aberto com a função hash auxiliar $h'(k) = k$. Ilustre o resultado da inserção dessas chaves utilizando sondagem linear, utilizando sondagem quadrática com $c_1 = 1$ e $c_2 = 3$, e utilizando hash duplo com $h_1(k) = k$ e $h_2(k) = 1 + (k \bmod (m - 1))$.
- 11.4-2** Escreva pseudocódigo para `HASH-DELETE` como descrito no texto e modifique `HASH-INSERT` para manipular o valor especial `DELETED`.
- 11.4-3** Considere uma tabela de espalhamento de endereços abertos com hashing uniforme. Dê limites superiores para o número esperado de sondagens em uma busca mal sucedida e para o número de sondagens em uma busca bem sucedida quando o fator de carga for $3/4$ e quando for $7/8$.

11.4-4 ★ Suponha que utilizamos hash duplo para resolver colisões; isto é, usamos a função hash $h(k, i) = (h_1(k) + ih_2(k)) \bmod m$. Mostre que, se m e $h_2(k)$ têm máximo divisor comum $d \geq 1$ para alguma chave k , então uma busca mal sucedida para a chave k examina $(1/d)$ -ésimo da tabela de espalhamento antes de retornar à posição $h_1(k)$. Assim, quando $d = 1$, de modo que m e $h_2(k)$ são primos entre si, a busca pode examinar a tabela de espalhamento inteira. (*Sugestão:* Consulte o Capítulo 31.)

11.4-5 ★ Considere uma tabela de espalhamento de endereço aberto com um fator de carga α . Encontre o valor não zero α para o qual o número esperado de sondagens em uma busca mal sucedida é igual a duas vezes o número esperado de sondagens em uma busca bem sucedida. Use os limites superiores dados pelos Teoremas 11.6 e 11.8 para esses números esperados de sondagens.

11.5 ★ HASHING PERFEITO

Embora, muitas vezes, seja uma boa escolha por seu excelente desempenho no caso médio, o hashing também pode proporcionar excelente desempenho *no pior caso*, quando o conjunto de chaves é **estático**: assim que as chaves são armazenadas na tabela, o conjunto de chaves nunca muda. Algumas aplicações têm naturalmente conjuntos de chaves estáticos: considere o conjunto de palavras reservadas em uma linguagem de programação ou o conjunto de nomes de arquivos em um CD-ROM. Damos a uma técnica de hashing o nome de **hashing perfeito** se forem exigidos $O(1)$ acessos à memória para executar uma busca no pior caso.

Para criar um esquema de hashing perfeito, usamos dois níveis de aplicação do hash, com hashing universal em cada nível. A Figura 11.6 ilustra essa abordagem.

O primeiro nível é essencialmente o mesmo do hashing com encadeamento: as n chaves são espalhadas por m posições utilizando uma função hash h cuidadosamente selecionada de uma família de funções hash universal.

Porém, em vez de fazer uma lista ligada das chaves espalhadas para a posição j , usamos uma pequena **tabela de hashing secundário** S_j com uma função hash associada h_j . Escolhendo as funções hash h_j cuidadosamente, podemos garantir que não haverá colisões no nível secundário.

Contudo, para garantir que não haverá nenhuma colisão no nível secundário, é preciso que o tamanho m_j da tabela de espalhamento S_j seja o quadrado do número n_j de chaves que se espalham para a posição j . Embora pareça provável que a dependência quadrática de m_j em relação a n_j torne excessivo o requisito global de armazenamento, mostraremos que, escolhendo bem a função hash de primeiro nível, podemos limitar a quantidade total de espaço usado esperado a $O(n)$.

Usamos funções hash escolhidas das classes universais de funções hash da Seção 1.3.3. A função hash de primeiro nível vem da classe p_m onde, como na Seção 1.3.3, p é um número primo maior que qualquer valor de chave. Essas chaves espalhadas para a posição j são espalhadas mais uma vez para uma tabela de espalhamento secundário S_j de tamanho m_j com a utilização de uma função hash h_j escolhida na classe p_{mj^1} .

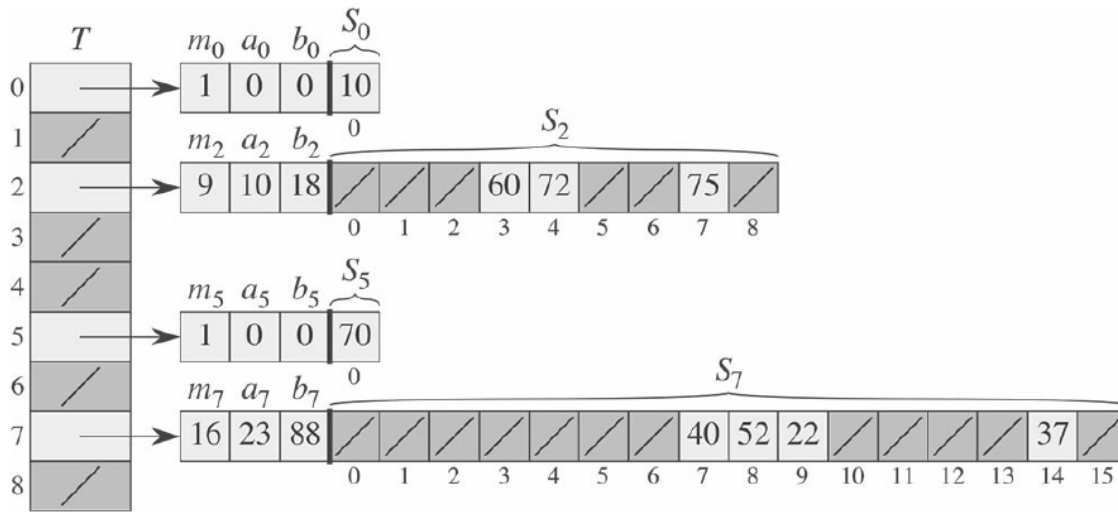


Figura 11.6 Utilização de hash perfeito para armazenar o conjunto $K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$. A função hash externa é $h(k) = ((ak + b) \bmod p) \bmod m$, onde $a = 3$, $b = 42$, $p = 101$, e $m = 9$. Por exemplo, $h(75) = 2$, e portanto os hashes da chave 75 vão para o espaço vazio 2 da tabela T . Uma tabela de hash secundária S_j armazena todas as chaves cujos hashes vão para o espaço vazio j . O tamanho da tabela de hash S é $n_j = n_2$, e a função hash associada é $h_j(k) = ((a_jk + b_j) \bmod p) \bmod n_j$. Visto que $h_2(75) = 7$, a chave 75 é armazenada no espaço vazio 7 da tabela de hash secundária S_2 . Não ocorre nenhuma colisão em qualquer das tabelas de hash secundárias e, portanto, a busca leva tempo constante no pior caso.

Prosseguiremos em duas etapas. Primeiro, determinaremos como assegurar que as tabelas secundárias não tenham nenhuma colisão. Em segundo lugar, mostraremos que a quantidade global esperada de memória utilizada para a tabela de espalhamento primário e para todas as tabelas de espalhamento secundário é $O(n)$.

Teorema 11.9

Suponha que armazenamos n chaves em uma tabela de espalhamento de tamanho $m = n_2$ usando uma função hash h escolhida aleatoriamente de uma classe universal de funções hash. Então, a probabilidade de haver quaisquer colisões é menor que $1/2$.

Prova Há pares de chaves que podem colidir; cada par colide com probabilidade $1/m$ se h é escolhida aleatoriamente de uma família universal de funções hash. Seja X uma variável aleatória que conta o número de colisões. Quando $m = n_2$, o número esperado de colisões é

$$\begin{aligned} E[X] &= \binom{n}{2} \cdot \frac{1}{n^2} \\ &= \frac{n^2 - n}{2} \cdot \frac{1}{n^2} \\ &< 1/2. \end{aligned}$$

(Essa análise é semelhante à análise do paradoxo do aniversário na Seção 5.4.1.) A aplicação da desigualdade de Markov (C.30), $\Pr \{X \geq t\} \leq E[X]/t$, com $t = 1$ conclui a prova.

Na situação descrita no Teorema 11.9, onde $m = n_2$, decorre que uma função hash h escolhida aleatoriamente de tem maior probabilidade de não ter *nenhuma* colisão. Dado o conjunto K de n chaves às quais o hash deve ser aplicado (lembre-se de que K é estático), é fácil encontrar uma função hash h livre de colisões com algumas tentativas aleatórias.

Contudo, quando n é grande, uma tabela de espalhamento de tamanho $m = n_2$ é excessiva. Assim, adotamos a abordagem de hash de dois níveis e usamos a abordagem do Teorema 11.9 apenas para o hash das entradas dentro de cada posição. Usamos uma função hash h exterior, ou de primeiro nível, para espalhar as chaves para $m = n$ posições. Então, se n_j chaves são espalhadas para a posição j , usamos uma tabela de espalhamento secundário S_j de tamanho $m_j = n_j^2$ para busca de tempo constante livre de colisões.

Agora, trataremos da questão de assegurar que a memória global usada seja $O(n)$. Como o tamanho m_j da j -ésima tabela de espalhamento secundário cresce quadraticamente com o número n_j de chaves armazenadas, existe o risco de a quantidade global de armazenamento ser excessiva.

Se o tamanho da tabela de primeiro nível é $m = n$, então a quantidade de memória usada é $O(n)$ para a tabela de espalhamento primário, para o armazenamento dos tamanhos m_j das tabelas de espalhamento secundário e para o armazenamento dos parâmetros a_j e b_j que definem as funções hash secundário h_j extraídas da classe p, m da Seção 11.3.3 (exceto quando $n_j = 1$ e usamos $a = b = 0$). Os teorema e corolário seguintes fornecem um limite para os tamanhos combinados esperados de todas as tabelas de espalhamento secundário. Um segundo corolário limita a probabilidade de o tamanho combinado de todas as tabelas de espalhamento secundário ser superlinear. (Na verdade, equivale ou excede $4n$).

Teorema 11.10

Suponha que armazenamos n chaves em uma tabela de espalhamento de tamanho $m = n$ usando uma função hash h escolhida aleatoriamente de uma classe universal de funções hash. Então, temos

$$E \left[\sum_{j=0}^{m-1} n_j^2 \right] < 2n,$$

onde n_j é o número de chaves que efetuam hash para a posição j .

Prova Começamos com a identidade a seguir, válida para qualquer inteiro não negativo a :

$$a^2 = a + 2 \binom{a}{2}. \quad (11.6)$$

Temos

$$\begin{aligned}
& E \left[\sum_{j=0}^{m-1} n_j^2 \right] \\
&= E \left[\sum_{j=0}^{m-1} \left(n_j + 2 \binom{n_j}{2} \right) \right] \quad (\text{pela equação (11.6)}) \\
&= E \left[\sum_{j=0}^{m-1} n_j \right] + 2E \left[\sum_{j=0}^{m-1} \binom{n_j}{2} \right] \quad (\text{por linearidade da esperança}) \\
&= E[n] + 2 E \left[\sum_{j=0}^{m-1} \binom{n_j}{2} \right] \quad (\text{pela equação (11.1)}) \\
&= n + E \left[\sum_{j=0}^{m-1} \binom{n_j}{2} \right] \quad (\text{já que } n \text{ não é uma variável aleatória})
\end{aligned}$$

$$\sum_{j=0}^{m-1} \binom{n_j}{2}$$

Para avaliar o somatório $\sum_{j=0}^{m-1} \binom{n_j}{2}$, observamos que ele é simplesmente o número total de pares de chaves na tabela de espalhamento que colidem. Pelas propriedades de hash universal, o valor esperado desse somatório é, no máximo,

$$\begin{aligned}
\binom{n}{2} \frac{1}{m} &= \frac{n(n-1)}{2m} \\
&= \frac{n-1}{2},
\end{aligned}$$

já que $m = n$. Assim,

$$\begin{aligned}
E \left[\sum_{j=0}^{m-1} n_j^2 \right] &\leq n + 2 \frac{n-1}{2} \\
&= 2n - 1 \\
&< 2n.
\end{aligned}$$

Corolário 11.11

Suponha que armazenamos n chaves em uma tabela de espalhamento de tamanho $m = n$ usando uma função hash h escolhida aleatoriamente de uma classe universal de funções hash e definimos o tamanho de cada tabela de espalhamento secundário como $m_j = n^{2j}$ para $j = 0, 1, \dots, m-1$. Então, a quantidade esperada de armazenamento exigido para todas as tabelas de espalhamento secundário em um esquema de hash perfeito é menor que $2n$.

Prova Visto que $m_j = n^{2j}$ para $j = 0, 1, \dots, m-1$, o Teorema 11.10 nos dá

$$\begin{aligned}
E \left[\sum_{j=0}^{m-1} m_j \right] &= E \left[\sum_{j=0}^{m-1} n^{2j} \right] \\
&< 2n.
\end{aligned} \tag{11.7}$$

o que conclui a prova.

Corolário 11.12

Suponha que armazenamos n chaves em uma tabela de espalhamento de tamanho $m = n$, usando uma função hash h escolhida aleatoriamente de uma classe universal de funções hash e definimos o tamanho de cada tabela de espalhamento secundário como $m_j = n^{2^j}$ para $j = 0, 1, \dots, m-1$. Então, a probabilidade de o armazenamento total usado para tabelas de espalhamento secundário ser igual ou exceder $4n$ é menor que $1/2$.

Prova Aplicamos mais uma vez a desigualdade de Markov (C.30), $\Pr\{X \geq t\} \leq E[X]/t$, desta vez à desigualdade (11.7), com $X = \sum_{j=0}^{m-1} m_j$ e $t = 4n$:

$$\begin{aligned} \Pr\left\{\sum_{j=0}^{m-1} m_j \geq 4n\right\} &\leq \frac{E\left[\sum_{j=0}^{m-1} m_j\right]}{4n} \\ &< \frac{2n}{4n} \\ &= 1/2. \end{aligned}$$

Pelo Corolário 11.12, vemos que, se testarmos algumas funções hash escolhidas aleatoriamente de uma família universal, encontraremos rapidamente uma função que utiliza uma quantidade razoável de espaço de armazenamento.

Exercícios

11.5-1 ★ Suponha que inserimos n chaves em uma tabela de espalhamento de tamanho m usando o endereçamento aberto e hash uniforme. Seja $p(n, m)$ a probabilidade de não ocorrer nenhuma colisão. Mostre que $p(n, m) \leq e^{-n/(m-1)/2m}$. (Sugestão: Consulte a equação (3.12).) Demonstre que, quando n excede $\sqrt{m}$, a probabilidade de evitar colisões cai rapidamente a zero.

Problemas

11-1 Limite para sondagem mais longa em hashing

Suponha que usamos uma tabela de espalhamento de endereçamento aberto de tamanho m para armazenar $n \leq m/2$ itens.

- Considerando hashing uniforme, mostre que, para $i = 1, 2, \dots, n$, a probabilidade de a i -ésima inserção exigir estritamente mais de k sondagens é, no máximo, 2^{-k} .
- Mostre que, para $i = 1, 2, \dots, n$, a probabilidade de a i -ésima inserção exigir mais de $2 \lg n$ sondagens é, no máximo, $O(1/n^2)$.

Seja X_i a variável aleatória que denota o número de sondagens exigidas pela i -ésima inserção. Você mostrou na parte (b) que $\Pr\{X_i > 2 \lg n\} \leq O(1/n^2)$. Seja $X = \max_{1 \leq i \leq n} X_i$ a variável aleatória que denota o número máximo de sondagens exigidas por quaisquer das n inserções.

- Mostre que $\Pr\{X > 2 \lg n\} \leq O(1/n)$.

d. Mostre que o comprimento esperado EX da sequência de sondagens mais longa é $O(\lg n)$.

11-2 Limite do tamanho por posição no caso de encadeamento

Suponha que temos uma tabela de espalhamento com n posições, com colisões resolvidas por encadeamento e que n chaves sejam inseridas na tabela. Cada chave tem igual probabilidade de ter hash para cada posição. Seja M o número máximo de chaves em qualquer posição após todas as chaves terem sido inseridas. Sua missão é provar um limite superior $O(\lg n / \lg \lg n)$ para $E[M]$, o valor esperado de M .

a. Mostre que a probabilidade Q_k de ocorrer hash de exatamente k chaves para uma determinada posição é dada por

$$Q_k = \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{n-k} \binom{n}{k}.$$

b. Seja P_k a probabilidade de $M = k$, isto é, a probabilidade de a posição que contém o número máximo de chaves conter k chaves. Mostre que $P_k \leq nQ_k$.

c. Use a aproximação de Stirling, equação (3.18), para mostrar que $Q_k < e^k / k^k$.

d. Mostre que existe uma constante $c > 1$ tal que $Q_{k_0} < 1/n^3$ para $k_0 = c \lg n / \lg \lg n$. Conclua que $P_k < 1/n^2$ para $k \geq k_0 = c \lg n / \lg \lg n$.

e. Mostre que

$$E[M] < \Pr\left\{M > \frac{c \lg n}{\lg \lg n}\right\} \cdot n + \Pr\left\{M \leq \frac{c \lg n}{\lg \lg n}\right\} \cdot \frac{c \lg n}{\lg \lg n}.$$

Conclua que $E[M] = O(\lg n / \lg \lg n)$.

11-3 Sondagem quadrática

Suponha que recebemos uma chave k para buscar numa tabela de espalhamento com posições $0, 1, \dots, m-1$, e suponha que temos uma função hash h que mapeia o espaço de chaves para o conjunto $\{0, 1, \dots, m-1\}$. O esquema de busca é o seguinte:

1. Calcule o valor $j = h(k)$ e faça $i = 0$.
2. Sonde a posição j em busca da chave desejada k . Se a encontrar ou se essa posição estiver vazia, encerre a busca.
3. Faça $j = i + 1$. Se agora i for igual a m , a tabela está cheia; portanto, encerre a busca. Caso contrário, defina $j = (i + j) \bmod m$ e retorne à etapa 2.

Suponha que m é uma potência de 2.

- a. Mostre que esse esquema é uma instância do esquema geral de “sondagem quadrática”, exibindo as constantes c_1 e c_2 adequadas para a equação (11.5).
- b. Prove que esse algoritmo examina cada posição da tabela no pior caso.

Seja uma classe de funções hash na qual cada função hash $h \in H$ mapeia o universo U de chaves para $\{0, 1, \dots, m-1\}$. Dizemos que H é ***k-universal*** se, para toda sequência fixa de k chaves distintas $\langle x_{(1)}, x_{(2)}, \dots, x_{(k)} \rangle$ e para qualquer h escolhido aleatoriamente de H , a sequência $\langle h(x_{(1)}), h(x_{(2)}), \dots, h(x_{(k)}) \rangle$ tem igual probabilidade de ser qualquer uma das m_k sequências de comprimento k com elementos extraídos de $\{0, 1, \dots, m-1\}$.

- Mostre que, se a família H de funções hash é 2-universal, então ela é universal.
- Suponha que o universo U seja o conjunto de ênuplas de valores extraídos de $\{0, 1, \dots, p-1\}$, onde p é primo. Considere um elemento $x = \langle x_0, x_1, \dots, x_{n-1} \rangle \in U$. Para qualquer ênupla $a = \langle a_0, a_1, \dots, a_{n-1} \rangle \in U$, defina a função hash h_a por

$$h_a(x) = \left(\sum_{j=0}^{n-1} a_j x_j \right) \bmod p$$

Seja $H = \{h_a\}$. Mostre que H é universal, mas não 2-universal. (Sugestão: Encontre uma chave para a qual todas as funções hash em H produzem o mesmo valor.)

- Suponha que modificamos ligeiramente H em relação à parte (b): para qualquer $a \in U$ e para qualquer $b \in \mathbb{Z}_p$, defina

$$h'_{ab}(x) = \left(\sum_{j=0}^{n-1} a_j x_j + b \right) \bmod p$$

e $H' = \{h'_{ab}\}$. Demonstre que H' é 2-universal. (Sugestão: Considere ênuplas fixas $x \in U$ e $y \in U$, com $x_i \neq y_i$ para alguns i . O que acontece com $h'_{ab}(x)$ e $h'_{ab}(y)$ à medida que a_i e b percorrem $\mathbb{Z}_p$?)

- Suponha que Alice e Bob concordam secretamente com uma função hash h de uma família 2-universal de funções hash. Cada $h \in H$ mapeia de um universo de chaves U para $\mathbb{Z}_p$, onde p é primo. Mais tarde, Alice envia uma mensagem m a Bob pela Internet, na qual $m \in U$. Ela autentica essa mensagem para Bob também enviando uma marca de autenticação $t = h(m)$, e Bob verifica se o par (m, t) que ele recebe satisfaz $t = h(m)$. Suponha que um adversário intercepte (m, t) em trânsito e tente enganar Bob substituindo o par (m, t) que recebeu por um par diferente (m, t) . Mostre que a probabilidade de o adversário conseguir enganar Bob e fazê-lo aceitar (m, t) é, no máximo, $1/p$, independentemente da capacidade de computação que o adversário tenha e até mesmo de o adversário conhecer a família de funções hash usada.

NOTAS DO CAPÍTULO

Knuth [211] e Gonnet [145] são excelentes referências para a análise de algoritmos de hashing. Knuth credita a H. P. Luhn (1953) a criação de tabelas de espalhamento, juntamente com o método de encadeamento para resolver colisões. Aproximadamente na mesma época, G. M. Amdahl apresentou a ideia do endereçamento aberto.

Carter e Wegman introduziram a noção de classes universais de funções hash em 1979 [58]. Fredman, Komlós e Szemerédi [112] desenvolveram o esquema de hashing perfeito para conjuntos estáticos apresentado na Seção 11.5.

Uma extensão de seu método para conjuntos dinâmicos, tratamento de inserções e eliminações em tempo esperado amortizado $O(1)$, foi apresentada por Dietzfelbinger *et al.* [87].

[†] Quando $n = m = 1$, não precisamos realmente de uma função hash para a posição j ; quando escolhemos uma função hash $h(k) = ((ak + b) \bmod p) \bmod m$ para tal posição, usamos simplesmente $a = b = 0$.